

API Testing & Web Services

Complete Study Material

Postman · Playwright · CI/CD · REST · JSON · HTTP

#	Topic	Key Concepts
01	Web Applications	Types, Architecture, Overview
02	Web URL	Structure, Syntax, Components
03	JSON	Format, Characteristics, Usage
04	API & Web Services	SOA, REST, SOAP, Types
05	HTTP Structure	Request/Response, Methods, Status Codes
06	Postman	CRUD, Test Cases, Auth, Reports
07	Playwright API Testing	Setup, CRUD, Parameters
08	Validations & Assertions	Status, Body, Headers, Time
09	Authentication	Basic, Bearer, OAuth 2.0, API Keys
10	Test Data & CI/CD	Chaining, Variables, Integration

Chapter 1 · Introduction to Web Applications

1.1 Types of Applications

Modern software applications can be classified based on their deployment model, user interaction, and architecture. Understanding these types is fundamental before diving into API testing.

Type	Description	Examples
Desktop App	Runs locally on the user's machine; OS-dependent.	MS Word, Photoshop
Web App	Runs in a browser; server-side or client-side rendering.	Gmail, GitHub, Jira
Mobile App	Native or hybrid apps on iOS/Android devices.	WhatsApp, Uber
Cloud App	Hosted entirely on cloud infrastructure; SaaS model.	Salesforce, AWS Console
PWA	Progressive Web Apps — web apps with offline & native-like capabilities.	Twitter Lite, Starbucks
Hybrid App	Combination of native and web technologies (e.g. Cordova).	Ionic apps, Flipkart

1.2 Architecture of a Web Application

Web applications typically follow a layered architectural model. The most common patterns are the 3-tier architecture and the Microservices architecture.

Three-Tier Architecture

- **Presentation Tier (Frontend):** The user interface layer — HTML, CSS, JavaScript, frameworks like React or Angular. It sends HTTP requests to the backend.
- **Application / Logic Tier (Backend):** Business logic layer — Node.js, Java Spring, Python Django, etc. Exposes APIs consumed by the frontend.
- **Data Tier (Database):** Stores and retrieves data — MySQL, PostgreSQL, MongoDB, Redis, etc.

Microservices Architecture

In microservices, each business capability is a separate, independently deployable service that communicates via APIs (usually REST or gRPC). Advantages include independent scaling, technology heterogeneity, and fault isolation.

Key Insight: In API testing, we primarily test the Application Tier — the business logic exposed through HTTP endpoints. Understanding the full architecture helps testers identify the right test boundaries.

Chapter 2 · Web URL

2.1 What is a URL?

A **Uniform Resource Locator (URL)** is the address used to access resources on the internet. Every web page, API endpoint, image, or file accessible over the web has a unique URL. URLs are a subset of URIs (Uniform Resource Identifiers).

URL vs URI: A URI is a broader concept — it identifies a resource. A URL is a URI that also provides the means to locate the resource (i.e., includes a protocol and address).

2.2 Syntax of URL

A URL is composed of several parts, each playing a distinct role:

```
scheme://userinfo@host:port/path?query#fragment
```

Example:

```
https://api.example.com:8080/v1/users?role=admin&active;=true#section2
```

Component	Description	Example Value
Scheme	Protocol used to access the resource.	https, http, ftp
Userinfo	Optional username/password (rarely used in modern APIs).	user:pass@
Host	Domain name or IP address of the server.	api.example.com
Port	TCP port number; defaults to 80 (HTTP) or 443 (HTTPS).	:8080
Path	Hierarchical location of the resource on the server.	/v1/users
Query String	Key=value pairs for filtering or parameterizing requests.	?role=admin&active;=true
Fragment	Client-side anchor reference; not sent to server.	#section2

URL Encoding

URLs can only contain certain characters. Special characters must be percent-encoded. For example, a space becomes **%20**, and an ampersand in a value becomes **%26**. API testing tools like Postman handle encoding automatically.

Chapter 3 · JSON — JavaScript Object Notation

3.1 What is JSON?

JSON (JavaScript Object Notation) is a lightweight, human-readable, text-based data interchange format. Originally derived from JavaScript, it is now language-independent and is the de facto standard for API request/response bodies. JSON is defined by RFC 8259.

JSON Data Types

- **String** — Unicode text enclosed in double quotes: "Hello"
- **Number** — Integer or floating point: 42, 3.14
- **Boolean** — true or false
- **Null** — null (represents absence of a value)
- **Array** — Ordered list of values: [1, 2, 3]
- **Object** — Unordered set of key/value pairs: {"key": "value"}

3.2 JSON Example

```
{
  "id": 101,
  "name": "Alice Johnson",
  "email": "alice@example.com",
  "active": true,
  "roles": ["admin", "tester"],
  "address": {
    "city": "Bangalore",
    "pincode": "560001"
  },
  "lastLogin": null
}
```

3.3 Characteristics of JSON

Characteristic	Description
Lightweight	Minimal syntax overhead; smaller payload size compared to XML.
Human-readable	Easy to read, write, and debug without special tools.
Language-independent	Supported by all major languages: Python, Java, JS, C#, etc.
Self-describing	Key names provide context about the data they hold.
Hierarchical	Supports nested objects and arrays for complex data structures.
Strict syntax	Keys must be strings in double quotes; trailing commas not allowed.

No comments	JSON spec does not support comments (use JSONC for dev configs).
-------------	--

JSON vs XML: JSON is generally preferred for REST APIs due to its smaller size and simpler structure. XML remains common in SOAP web services and enterprise integrations.

Chapter 4 · API and Web Services

4.1 Layers of Web Application / SOA

Service-Oriented Architecture (SOA) organizes software as a collection of interoperable services. Testing a web application requires understanding each layer:

Layer	Testing Scope
UI / Presentation Layer	Tests: UI testing (Selenium, Playwright), usability testing.
Business / API Layer	Tests: API testing (Postman, Rest Assured, Playwright API).
Service / Integration Layer	Tests: Integration testing, contract testing.
Data / Database Layer	Tests: Database testing, data validation, SQL queries.

4.2 Introduction to API

An **API (Application Programming Interface)** is a set of defined rules and protocols that allow different software applications to communicate with each other. APIs abstract the underlying implementation and expose only the objects or actions the developer needs.

Analogy: Think of an API as a waiter in a restaurant. You (the client) tell the waiter (API) what you want, and the waiter fetches it from the kitchen (server) and brings it back to you.

4.3 Types of API Testing

Functional Testing: Verify that the API performs its intended function correctly.

Load / Performance Testing: Assess behavior under high traffic or stress conditions.

Security Testing: Check for vulnerabilities: injection, broken auth, data exposure.

Contract Testing: Ensure the API conforms to the agreed-upon contract/schema.

Integration Testing: Validate that the API integrates correctly with other services.

Regression Testing: Ensure new changes haven't broken existing functionality.

Negative Testing: Send invalid data and verify appropriate error responses.

Fuzz Testing: Send random/unexpected data to discover edge cases and crashes.

Chapter 5 · API and Web Services (Continued)

5.1 Introduction to Web Services

A **Web Service** is a standardized method for propagating messages between client and server applications on the World Wide Web. It is a type of API that communicates over the HTTP protocol and follows specific standards such as REST or SOAP.

REST vs SOAP

Feature	REST	SOAP
Protocol	HTTP/HTTPS	HTTP, SMTP, TCP
Data Format	JSON, XML, HTML	XML only
Standards	Lightweight guidelines	WS-* strict standards
Performance	Faster, less overhead	Heavier, more overhead
Statefulness	Stateless	Can be stateful
Use Case	Public APIs, mobile	Enterprise, banking

5.2 Types of Web Service Testing

- **WSDL Validation:** Validate the WSDL (Web Services Description Language) document structure for SOAP services.
- **Endpoint Testing:** Verify that the correct service endpoint is accessible.
- **Request/Response Validation:** Confirm correct XML payloads are sent and received.
- **Security Testing:** WS-Security, TLS, authentication tokens.
- **Performance Testing:** Measure throughput, latency, and concurrency handling.
- **Interoperability Testing:** Test that the service works correctly across different platforms.

5.3 Advantages of Web Services

- ✓ Platform and language independent — any client can consume the service.
- ✓ Loosely coupled — services can change internally without affecting clients.
- ✓ Reusability — the same service can be used by multiple consumers.
- ✓ Interoperability — enables communication across different systems.
- ✓ Scalability — services can be independently scaled.
- ✓ Standardization — uses open standards (HTTP, JSON, XML, SSL).

Chapter 6 · HTTP Structure

6.1 HTTP Overview

HTTP (HyperText Transfer Protocol) is the foundation of data communication on the web. It is a request-response protocol where a client sends an HTTP request and the server returns an HTTP response. HTTP is stateless — each request is independent.

6.2 HTTP Request Structure

An HTTP request consists of four main parts:

```
METHOD /path HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer
{
  "username": "alice",
  "password": "secret123"
}
```

HTTP Methods

Method	Purpose	Use Case	Request Body
GET	Retrieve resource(s)	Read user list	No body
POST	Create a new resource	Create a user	Has body
PUT	Replace entire resource	Update user fully	Has body
PATCH	Partially update resource	Update user email	Has body
DELETE	Remove a resource	Delete a user	Optional body
HEAD	Same as GET but no body	Check if exists	No body
OPTIONS	Fetch supported methods	CORS preflight	No body

6.3 HTTP Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/json
X-Request-Id: abc-123
{
  "id": 42,
  "name": "Alice"
}
```

HTTP Status Code Categories

Range	Category	Common Codes
1xx	Informational	100 Continue, 101 Switching Protocols
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 304 Not Modified
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server Error	500 Internal Server Error, 503 Service Unavailable

Common Request Headers

- **Content-Type:** Describes the format of the request body (e.g., application/json).
- **Accept:** Tells the server what format the client can process in the response.
- **Authorization:** Contains credentials: Basic, Bearer token, API key.
- **Host:** The domain name of the server (required in HTTP/1.1).
- **User-Agent:** Identifies the client application making the request.
- **Cookie:** Sends stored cookies to the server for session management.

Chapter 7 · Postman

7.1 Overview of Postman

Postman is a popular API development and testing platform. It provides a graphical interface for constructing HTTP requests, organizing them into collections, writing test scripts in JavaScript, and generating reports. Postman supports REST, GraphQL, WebSocket, and SOAP.

Key Features

- Request builder with all HTTP methods, headers, body, and auth options.
- Collections — organize related API requests in folders.
- Environments — manage variables across dev/staging/production.
- Pre-request Scripts — execute JavaScript before sending a request.
- Tests (Post-response Scripts) — validate response using Postman test snippets.
- Collection Runner — execute all requests in a collection sequentially.
- Newman — Postman's command-line runner for CI/CD integration.
- Mock Servers — simulate API responses without a real backend.

7.2 Project 1: Basic CRUD Operations

CRUD stands for Create, Read, Update, Delete — the four fundamental operations of persistent storage, mapped to HTTP methods:

Operation	Method	Endpoint	Notes	Expected Status
Create	POST	/users	Send user data in JSON body	201 Created
Read	GET	/users/1	No body needed	200 OK
Update	PUT	/users/1	Send full updated object	200 OK
Partial	PATCH	/users/1	Send only changed fields	200 OK
Delete	DELETE	/users/1	No body needed	204 No Content

7.3 Project 2: Advanced Testing

How to Write API Test Cases

- **Identify Test Scope:** List all endpoints, methods, and scenarios.
- **Define Input:** Specify URL, method, headers, body, and auth.
- **Define Expected Output:** Status code, response body schema, and values.
- **Write Assertions:** Use Postman test snippets to validate expectations.
- **Handle Negative Cases:** Invalid inputs, missing fields, wrong auth.

Authentication & Authorization

```
// Basic Auth
pm.request.auth = { type: "basic", basic: [
  { key: "username", value: "admin" },
  { key: "password", value: "secret" }
] };

// Bearer Token
pm.request.headers.add({ key: "Authorization",
value: "Bearer " + pm.environment.get("token") });
```

Importance of Variables

- **Global Variables:** Shared across all collections and environments.
- **Environment Variables:** Scoped to a specific environment (dev, staging, prod).
- **Collection Variables:** Scoped to a single collection.
- **Local Variables:** Scoped to a single request's scripts.
- **Dynamic Variables:** Built-in values like {{\$guid}}, {{\$timestamp}}, {{\$randomEmail}}.

Test Validations Using Snippets

```
// Status code check
pm.test('Status is 200', () => {
  pm.response.to.have.status(200);
});

// Response body value
pm.test('Name is Alice', () => {
  const json = pm.response.json();
  pm.expect(json.name).to.eql('Alice');
});

// Response time
pm.test('Response < 500ms', () => {
  pm.expect(pm.response.responseTime).to.be.below(500);
});
```

Batch Execution & Report Generation

- **Collection Runner:** Run all requests in a collection with optional iteration count and data file.
- **Newman CLI:** Run collections from the command line: *newman run collection.json -e env.json*
- **HTML Reports:** Use newman-reporter-htmlextra to generate rich HTML test reports.
- **JUnit XML:** Use newman-reporter-junitfull for CI/CD integration with Jenkins/GitHub Actions.

Chapter 8 · Basic Playwright API Testing

8.1 Playwright APIRequestContext

Playwright provides the **APIRequestContext** class for making HTTP requests directly without launching a browser. It is ideal for API testing within the same test suite as E2E browser tests, sharing auth state, and intercepting network traffic.

Playwright's API testing is fully integrated with its assertion library, allowing the same expect() syntax for both UI and API tests — making it easy to chain UI and API checks in one test.

8.2 Creating Request Context

```
import { test, expect, request } from '@playwright/test';

// Global context (shared across tests)
const apiContext = await request.newContext({
  baseURL: 'https://api.example.com',
  extraHTTPHeaders: {
    'Accept': 'application/json',
    'Authorization': `Bearer ${process.env.TOKEN}`
  }
});
```

8.3 CRUD Operations Using Playwright

GET — Read

```
const response = await request.get('/users/1');
expect(response.ok()).toBeTruthy();
const user = await response.json();
expect(user.name).toBe('Alice');
```

POST — Create

```
const response = await request.post('/users', {
  data: { name: 'Bob', email: 'bob@example.com' }
});
expect(response.status()).toBe(201);
```

PUT — Update

```
const response = await request.put('/users/1', {
  data: { name: 'Bob Updated', email: 'bob@new.com' }
});
expect(response.ok()).toBeTruthy();
```

DELETE — Remove

```
const response = await request.delete('/users/1');
expect(response.status()).toBe(204);
```

8.4 Passing Parameters

Headers

```
await request.get('/protected', {
  headers: {
    'Authorization': 'Bearer token123',
    'X-Custom-Header': 'value'
  }
});
```

Query Parameters

```
await request.get('/users', {
  params: { role: 'admin', active: 'true', page: '1' }
});
// Resolves to: /users?role=admin&active=true&page=1
```

Path Parameters

```
const userId = 42;
const response = await request.get(`/users/${userId}`);
// Resolves to: /users/42
```

Request Body

```
await request.post('/users', {
  data: { // JSON body (auto-serialized)
    name: 'Alice',
    age: 28
  }
});

// For form data:
await request.post('/upload', {
  form: { field1: 'value1', field2: 'value2' }
});
```

8.5 Handling API Responses

- **response.status()** — Returns the HTTP status code as a number.
- **response.ok()** — Returns true if status is in the 200–299 range.
- **response.json()** — Parses the body as JSON (returns a Promise).
- **response.text()** — Returns the body as a plain string.
- **response.headers()** — Returns all response headers as an object.
- **response.headersArray()** — Returns headers as an array of {name, value} objects.

- **response.body()** — Returns the raw body as a Buffer.

Chapter 9 · Validations & Assertions

9.1 Playwright expect Assertions

Playwright's **expect()** function provides a powerful assertion library for validating API responses. It supports synchronous and asynchronous assertions with clear error messages.

9.2 Validating Responses

Status Codes

```
expect(response.status()).toBe(200);
expect(response.ok()).toBeTruthy();
expect(response.status()).toBeGreaterThanOrEqual(200);
expect(response.status()).toBeLessThan(300);
```

Response Body Values

```
const body = await response.json();
// Exact match
expect(body.name).toBe('Alice');
// Contains check
expect(body.email).toContain('@example.com');
// Array length
expect(body.items).toHaveLength(5);
// Array contains item
expect(body.roles).toContain('admin');
// Object has property
expect(body).toHaveProperty('id');
expect(body).toHaveProperty('address.city', 'Bangalore');
// Type checking
expect(typeof body.id).toBe('number');
```

Response Headers

```
const headers = response.headers();
expect(headers['content-type']).toContain('application/json');
expect(headers['x-rate-limit-remaining']).toBeDefined();
```

Response Time

```
// Playwright doesn't expose response time natively;
// measure manually:
const start = Date.now();
const response = await request.get('/users');
const duration = Date.now() - start;
expect(duration).toBeLessThan(1000); // must be < 1 second
```

9.3 Partial Response Validation

Sometimes you only need to validate a subset of the response — especially useful when the response is large or contains dynamic fields like timestamps.

```
const body = await response.json();  
// toMatchObject – validates subset of properties  
expect(body).toMatchObject({  
  name: 'Alice',  
  active: true  
});  
// ignores extra fields like 'id', 'createdAt', etc.  
// Validate nested partial structure  
expect(body).toMatchObject({  
  address: { city: 'Bangalore' }  
});
```

Chapter 10 · Authentication & Authorization

Authentication verifies *who you are*; Authorization verifies *what you can do*. APIs commonly use several authentication mechanisms — each with different security characteristics.

10.1 Basic Authentication

Credentials (username:password) are Base64-encoded and sent in the Authorization header. Simple but insecure without HTTPS.

```
// Playwright
await request.get('/protected', {
  headers: {
    'Authorization': 'Basic ' + btoa('admin:secret123')
  }
});
```

10.2 Bearer Token Authentication

A token (JWT or opaque) is obtained from a login/auth endpoint and sent in the Authorization header as a Bearer token. This is the most common pattern for REST APIs.

```
// Step 1: Login to get token
const loginRes = await request.post('/auth/login', {
  data: { username: 'alice', password: 'pass123' }
});
const { token } = await loginRes.json();

// Step 2: Use token in subsequent requests
const profileRes = await request.get('/profile', {
  headers: { 'Authorization': `Bearer ${token}` }
});
```

10.3 OAuth 2.0

OAuth 2.0 is an authorization framework enabling third-party apps to obtain limited access without exposing credentials. Common grant types:

Grant Type	Use Case / Description
Authorization Code	For server-side apps; most secure; uses auth code exchange.
Client Credentials	For machine-to-machine (M2M) communication; no user context.
Implicit	For SPAs (deprecated in OAuth 2.1 — replaced by PKCE).
Password Grant	User credentials sent directly (deprecated; avoid in new apps).
PKCE	Authorization Code + Proof Key; secure for SPAs and mobile.

```
// Client Credentials flow (M2M)
const tokenRes = await request.post('/oauth/token', {
  form: {
    grant_type: 'client_credentials',
    client_id: process.env.CLIENT_ID,
    client_secret: process.env.CLIENT_SECRET,
    scope: 'read:users write:users'
  }
});
const { access_token } = await tokenRes.json();
```

10.4 API Key–Based Authentication

An API key is a unique identifier passed by the client. It can be sent via header, query parameter, or request body.

```
// Via header (most secure)
await request.get('/data', {
  headers: { 'X-API-Key': process.env.API_KEY }
});

// Via query parameter (visible in logs – avoid for sensitive APIs)
await request.get('/data', {
  params: { api_key: process.env.API_KEY }
});
```

Chapter 11 · Test Data Handling & CI/CD Integration

11.1 Static vs Dynamic Test Data

Aspect	Static Data	Dynamic Data
Definition	Fixed values in code or files.	Generated at runtime (timestamp, UUID).
Pros	Predictable, easy to debug.	Realistic, avoids data conflicts.
Cons	Can cause conflicts in shared envs.	Harder to verify exact values.
Examples	{ "name": "TestUser" }	faker.name(), Date.now()

11.2 Passing Data Between API Calls / Chaining

API chaining involves using the response of one API call as input to the next. This is essential for realistic test scenarios.

```
test('Create then fetch user', async ({ request }) => {
  // Step 1: Create user
  const createRes = await request.post('/users', {
    data: { name: 'Alice', email: 'alice@example.com' }
  });
  const { id } = await createRes.json();
  // Step 2: Fetch the created user using extracted ID
  const getRes = await request.get(`/users/${id}`);
  expect(getRes.ok()).toBeTruthy();
  const user = await getRes.json();
  expect(user.name).toBe('Alice');
});
```

11.3 Using Environment Variables

```
// playwright.config.ts
export default defineConfig({
  use: {
    baseURL: process.env.BASE_URL || 'https://api.staging.example.com',
    extraHTTPHeaders: {
      'Authorization': `Bearer ${process.env.API_TOKEN}`
    }
  }
});

// Run with:
// BASE_URL=https://api.prod.com npx playwright test
```

11.4 External Test Data (JSON Files)

```
// test-data/users.json
[
  { "name": "Alice", "email": "alice@example.com", "role": "admin" },
  { "name": "Bob", "email": "bob@example.com", "role": "user" }
]

// test file
import users from './test-data/users.json';

for (const user of users) {
  test(`Create user: ${user.name}`, async ({ request }) => {
    const res = await request.post('/users', { data: user });
    expect(res.status()).toBe(201);
  });
}
```

11.5 Integration with CI/CD

Integrating API tests into CI/CD pipelines ensures tests run automatically on every code change. Below is a GitHub Actions example:

```
# .github/workflows/api-tests.yml
name: API Tests
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
    with:
      node-version: '18'
      - run: npm ci
      - run: npx playwright install
      - run: npx playwright test --reporter=html
    env:
      BASE_URL: ${ secrets.API_BASE_URL }
      API_TOKEN: ${ secrets.API_TOKEN }
      - uses: actions/upload-artifact@v3
    with:
      name: playwright-report
      path: playwright-report/
```

CI/CD Best Practices

- ✓ Store sensitive data (tokens, passwords) in CI/CD secrets — never in code.
- ✓ Use separate environment configs for dev, staging, and production.
- ✓ Run smoke tests on every push; run full regression on merge to main.
- ✓ Gate deployments on test pass — block merges if API tests fail.

- ✓ Publish test reports as build artifacts for team visibility.
- ✓ Use parallel test execution to reduce pipeline duration.

Summary: This material has covered the complete landscape of API testing — from understanding web application architecture and HTTP fundamentals, through hands-on Postman and Playwright techniques, to authentication patterns, test data strategies, and CI/CD integration. Master each topic progressively and build real projects to solidify your understanding.